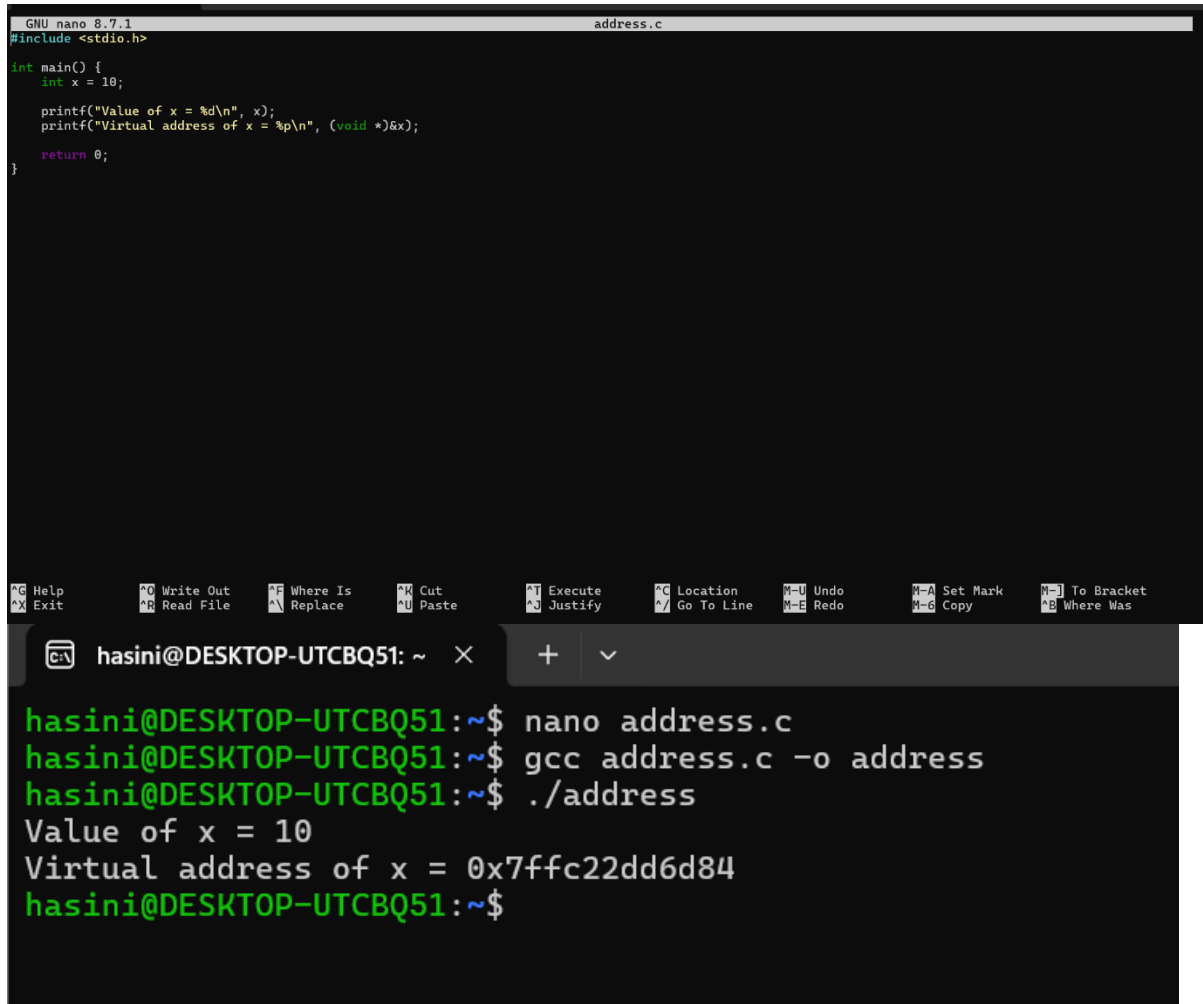


Name: S. Hasini
RollNo:2520030287

OSSP PRACTICAL

1.Virtual Address:



The screenshot shows a terminal window with a nano editor at the top editing a file named `address.c`. The code in the editor is as follows:

```
GNU nano 8.7.1 address.c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

Below the editor, the terminal shows the execution of the program:

```
hasini@DESKTOP-UTCBO51: ~$ nano address.c
hasini@DESKTOP-UTCBO51: ~$ gcc address.c -o address
hasini@DESKTOP-UTCBO51: ~$ ./address
Value of x = 10
Virtual address of x = 0x7ffc22dd6d84
hasini@DESKTOP-UTCBO51: ~$
```

The terminal output displays the value of `x` as 10 and its virtual address as `0x7ffc22dd6d84`.

2.Demand paging:

```
GNU nano 8.7.1 page.c *
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
hasini@DESKTOP-UTCQ51: ~$ nano page.c
hasini@DESKTOP-UTCQ51: ~$ gcc page.c -o page
hasini@DESKTOP-UTCQ51: ~$ ./page
Enter page number (0-3): 5
Page is already in memory.
Physical frame = 0
hasini@DESKTOP-UTCQ51: ~$ |
```

3.process spaces:

```
hasini@DESKTOP-UTCBSQ51: ~ × + v
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

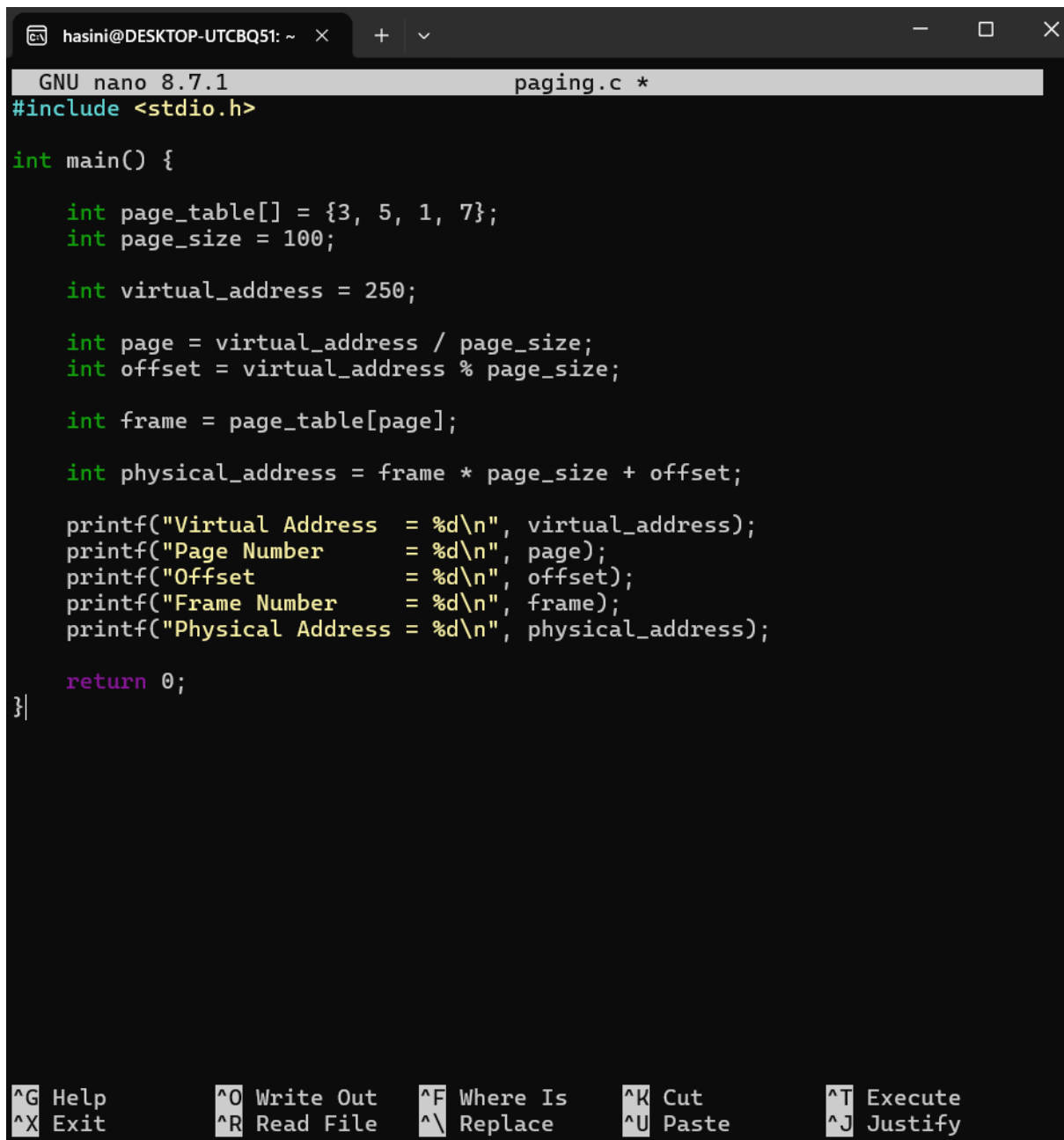


hasini@DESKTOP-UTCBO51: ~



```
hasini@DESKTOP-UTCBO51:~$ nano memory.c
hasini@DESKTOP-UTCBO51:~$ gcc memory.c -o memory
hasini@DESKTOP-UTCBO51:~$ ./memory
Code address : 0x5d8d2f5081a9
Data address : 0x5d8d2f50b010
BSS address  : 0x5d8d2f50b018
Heap address : 0x5d8d53248010
Stack address: 0x7ffe8deed34c
hasini@DESKTOP-UTCBO51:~$
```

4.address translation:



```
GNU nano 8.7.1          paging.c *
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address  = %d\n", virtual_address);
    printf("Page Number     = %d\n", page);
    printf("Offset          = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

Terminal window showing the nano editor interface. The window title is "hasini@DESKTOP-UTC BQ51: ~". The editor shows a C program named "paging.c" with the following code:

```
GNU nano 8.7.1          paging.c *
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address  = %d\n", virtual_address);
    printf("Page Number     = %d\n", page);
    printf("Offset          = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

The bottom status bar displays the following keyboard shortcuts:

Help	Write Out	Where Is	Cut	Execute
Exit	Read File	Replace	Paste	Justify

```
hasini@DESKTOP-UTCQ51: ~  
hasini@DESKTOP-UTCQ51:~$ nano paging.c  
hasini@DESKTOP-UTCQ51:~$ gcc paging.c -o paging  
hasini@DESKTOP-UTCQ51:~$ ./paging  
Virtual Address = 250  
Page Number     = 2  
Offset          = 50  
Frame Number    = 1  
Physical Address = 150  
hasini@DESKTOP-UTCQ51:~$ |
```

5. Malloc and free

```
GNU nano 8.7.1 malloc.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int *p;  
  
    p = malloc(5 * sizeof(int));  
  
    if (p == NULL) {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        p[i] = (i + 1) * 10;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        printf("%d ", p[i]);  
    }  
  
    printf("\n");  
  
    free(p);  
  
    return 0;  
}
```

```
hasini@DESKTOP-UTCQ51: ~$ nano malloc.c
hasini@DESKTOP-UTCQ51:~$ gcc malloc.c -o malloc
hasini@DESKTOP-UTCQ51:~$ ./malloc
10 20 30 40 50
hasini@DESKTOP-UTCQ51:~$ |
```

6.Segmentation Fault:

```
GNU nano 8.7.1 nullpointer.c *
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

hasini@DESKTOP-UTCQ51: ~

+

✓

```
hasini@DESKTOP-UTCQ51:~$ nano nullpointer.c
```

```
hasini@DESKTOP-UTCQ51:~$ gcc nullpointer.c -o nullpointer
```

```
hasini@DESKTOP-UTCQ51:~$
```

```
hasini@DESKTOP-UTCQ51:~$ ./nullpointer
```

```
Before accessing memory
```

```
Segmentation fault ./nullpointer
```

```
hasini@DESKTOP-UTCQ51:~$ |
```